A1

Ethan Tran

2026-02-17

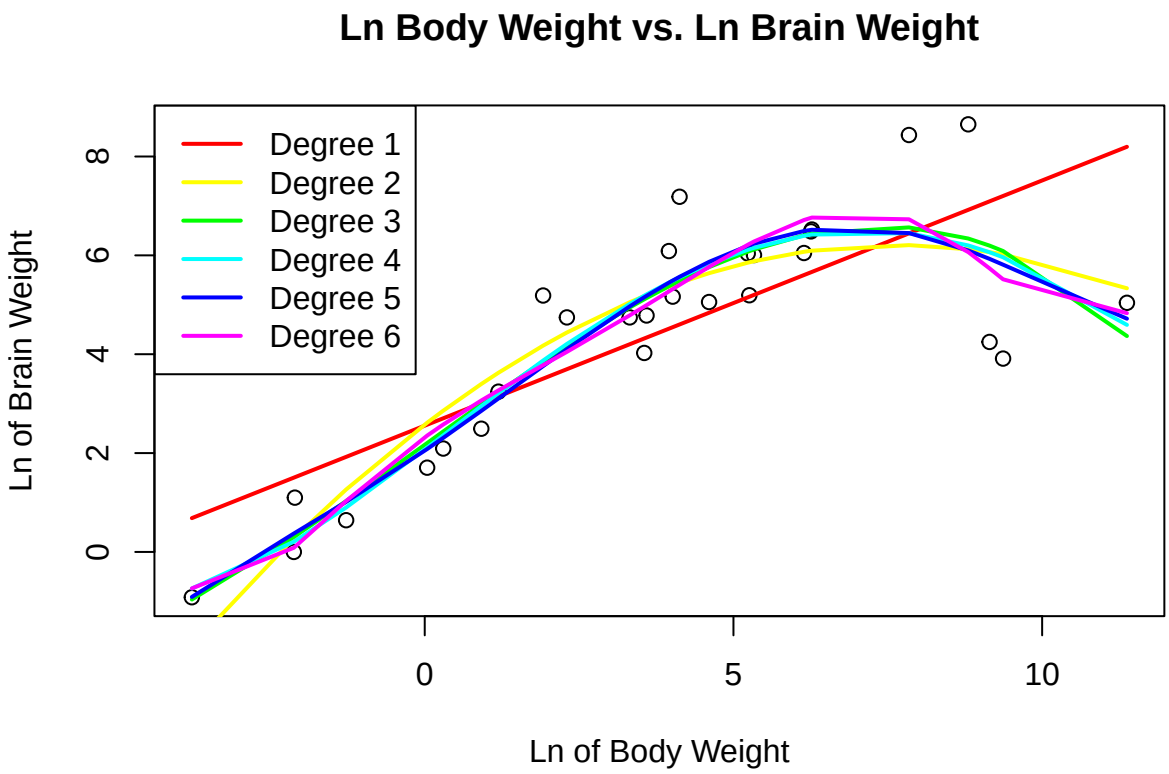
Q2.

a.

```r
library(MASS)
```

```
## Warning: package 'MASS' was built under R version 4.4.2
```

```r
data(Animals)

Animals$ln_body <- log(Animals$body)
Animals$ln_brain <- log(Animals$brain)

plot(Animals$ln_body, Animals$ln_brain,
     xlab = "Ln of Body Weight",
     ylab = "Ln of Brain Weight",
     main = "Ln Body Weight vs. Ln Brain Weight")

cols <- rainbow(6)

for (q in 1:6) {
  model <- lm(ln_brain ~ poly(ln_body, q), data = Animals)

  ord <- order(Animals$ln_body)

  lines(Animals$ln_body[ord], fitted(model)[ord], col = cols[q], lwd = 2)
}

legend("topleft", legend = paste("Degree", 1:6), col = cols, lwd = 2)
```


Ln Body Weight vs. Ln Brain Weight
Degree 1
Degree 2
Degree 3
Degree 4
Degree 5
Degree 6
Ln of Brain Weight
0
2
4
6
8
0
5
10
Ln of Body Weight

b.

```r
test_index <- c(2, 7, 11, 16, 26)
train <- Animals[-test_index, ]
test <- Animals[test_index, ]

APSE <- numeric(6)

for (q in 1:6) {
  model <- lm(ln_brain ~ poly(ln_body, q), data = train)
  y_pred <- predict(model, newdata = test)
  APSE[q] <- mean((test$ln_brain - y_pred)^2)
}

results <- data.frame(
  degree = 1:6,
  APSE = APSE
)

results
```

```
##   degree        APSE
## 1      1    6.999294
## 2      2    1.957592
## 3      3    1.858240
## 4      4    3.248757
## 5      5   56.157765
## 6      6 1217.380006
```

```r
which.min(APSE) # Model 3 has the lowest APSE
```

```
## [1] 3
```

c.

```r
ShuffledIndices = c(19, 8, 25, 5, 16, 3, 4, 17, 21, 10, 6, 27, 28, 20, 26, 23, 13, 14, 7, 15, 24, 2, 22
folds <- split(ShuffledIndices, rep(c(1, 2, 3, 4, 5), times = c(6, 6, 6, 5, 5)))
folds
```

```
## $`1`
## [1] 19  8 25  5 16  3
##
## $`2`
## [1]  4 17 21 10  6 27
##
## $`3`
## [1] 28 20 26 23 13 14
##
## $`4`
## [1]  7 15 24  2 22
##
## $`5`
## [1]  9 11 18 12  1
```

```r
cv_error <- numeric(6)

for (q in 1:6) {
  fold_APSE <- numeric(5)
  for (i in 1:5) {
    test_index <- folds[[i]]
    train_fold <- Animals[-test_index, ]
    test_fold <- Animals[test_index, ]

    model <- lm(ln_brain ~ poly(ln_body, q), data = train_fold)
    y_pred <- predict(model, newdata = test_fold)
    fold_APSE[i] <- mean((test_fold$ln_brain - y_pred)^2)
  }
  cv_error[q] <- mean(fold_APSE)
}

results <- data.frame(
  degree = 1:6,
  cv_error = cv_error
)

results
```

```
##   degree   cv_error
## 1      1   2.659165
## 2      2   1.653375
## 3      3   1.961035
## 4      4   5.057223
## 5      5  72.705664
## 6      6 165.774581
```

```r
which.min(cv_error) # Model 2 has the lowest APSE
```

```
## [1] 2
```

d.

The best model in part b. was degree 3 with APSE 1.858240.

The best model in part c. was degree 2 with APSE 1.653375.

They do not agree on the best polynomial degree.

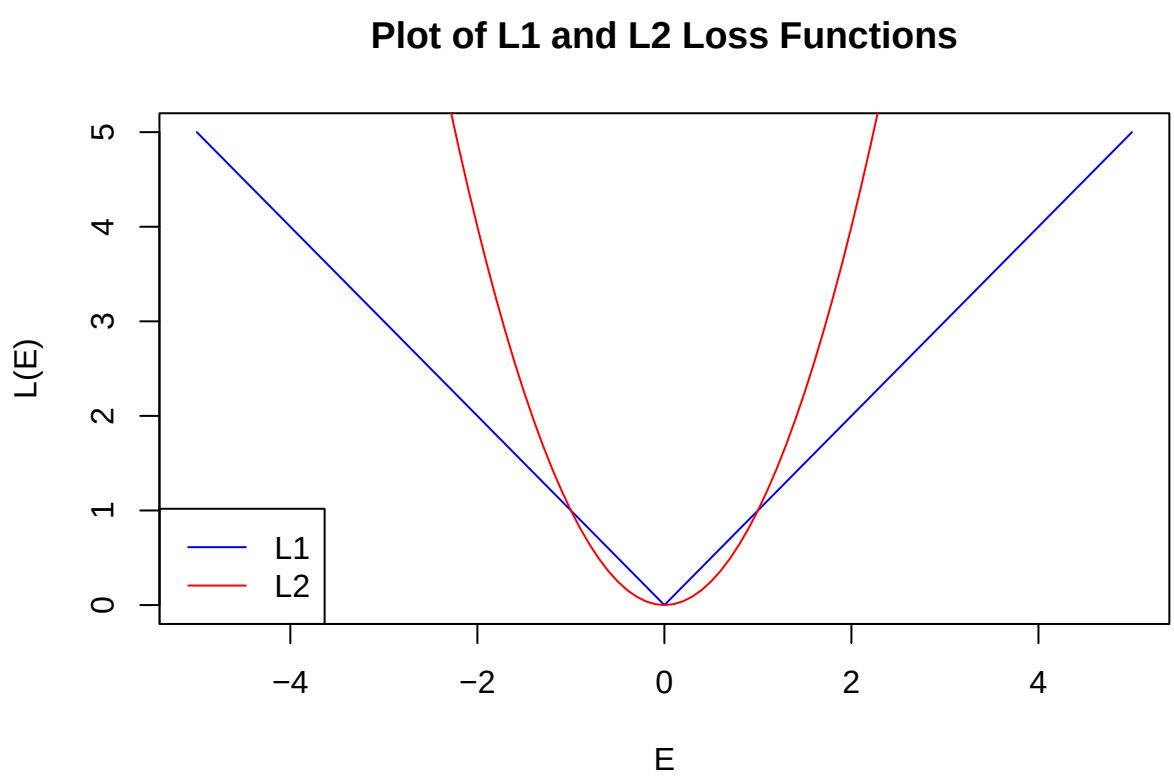
The result in part c. is more trustworthy as it uses more test sets through k-fold CV.

The model in part b. only uses 1 test set. This may result in overfitting if the training data is not a good representation of the dataset.

Q3.

a.

```r
l1_loss <- function(E) {
  return(abs(E))
}

l2_loss <- function(E) {
  return(E * E)
}

curve(l1_loss, from = -5, to = 5, col = "blue",
      main = "Plot of L1 and L2 Loss Functions", xlab = "E", ylab = "L(E)")
curve(l2_loss, from = -5, to = 5, col = "red", add = TRUE)
legend("bottomleft", legend = c("L1", "L2"), col = c("blue", "red"), lty = 1)
```



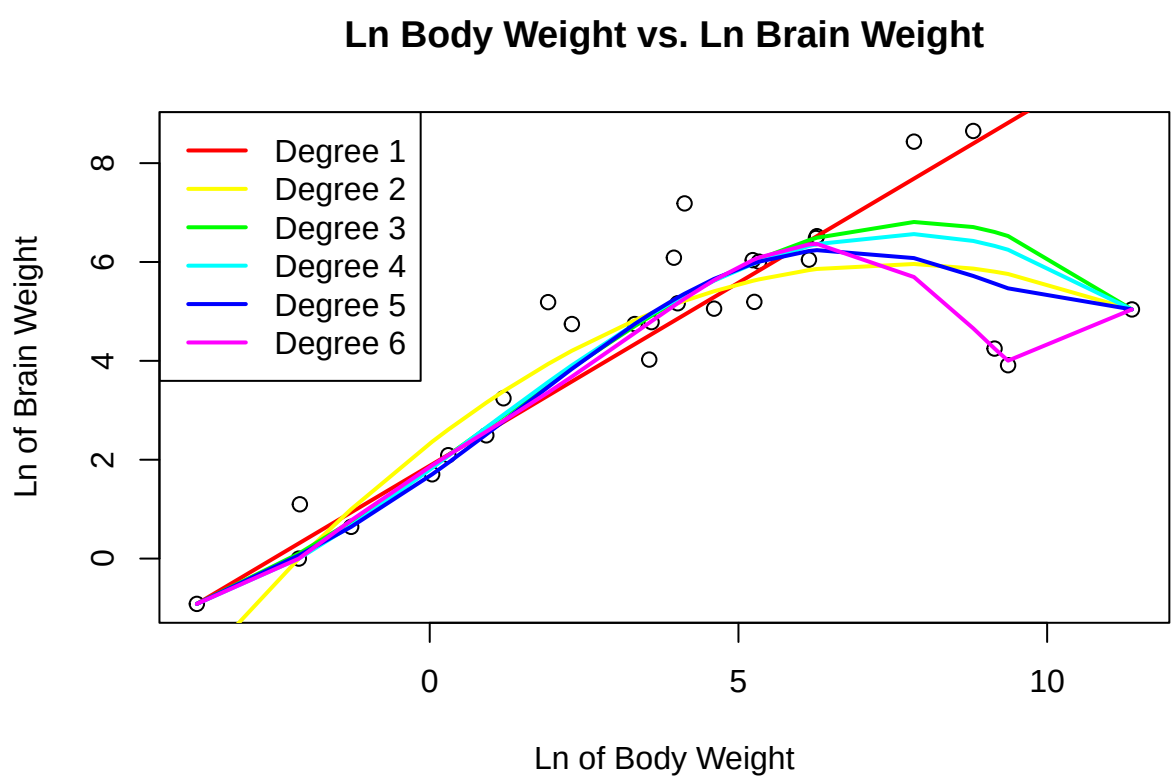
For smaller absolute prediction error (closer to true value), L1 loss penalizes more than L2. For larger absolute prediction error (further from true value), L2 loss penalizes more than L1.

b.

```r
library(L1pack)
```

```
## Loading required package: fastmatrix
```

```r
plot(Animals$ln_body, Animals$ln_brain,
     xlab = "Ln of Body Weight",
     ylab = "Ln of Brain Weight",
     main = "Ln Body Weight vs. Ln Brain Weight")

for (q in 1:6) {
  model <- lad(ln_brain ~ poly(ln_body, q), data = Animals)

  ord <- order(Animals$ln_body)

  lines(Animals$ln_body[ord], fitted(model)[ord], col = cols[q], lwd = 2)
}

legend("topleft", legend = paste("Degree", 1:6), col = cols, lwd = 2)
```



Higher degree models are more sensitive to outliers. Lower degree models are less sensitive to outliers.

c.

```r
test_index <- c(2, 7, 11, 16, 26)
train <- Animals[-test_index, ]
test <- Animals[test_index, ]

APAE <- numeric(6)

for (q in 1:6) {
  model <- lad(ln_brain ~ poly(ln_body, q), data = train)
  y_pred <- predict(model, newdata = test)
  APAE[q] <- mean(abs(test$ln_brain - y_pred))
}

results <- data.frame(
  degree = 1:6,
  APAE = APAE
)

results
```

```
##   degree      APAE
## 1      1  2.339059
## 2      2  1.883510
## 3      3  1.177508
## 4      4  2.090316
## 5      5  2.822778
## 6      6 17.991249
```

```r
which.min(APAE)
```

```
## [1] 3
```

Model 3 has the lowest APSE, which is the same result as in 2b.

d.

```r
cv_error <- numeric(6)

for (q in 1:6) {
  fold_APSE <- numeric(5)
  for (i in 1:5) {
    test_index <- folds[[i]]
    train_fold <- Animals[-test_index, ]
    test_fold <- Animals[test_index, ]

    model <- lad(ln_brain ~ poly(ln_body, q), data = train_fold)
    y_pred <- predict(model, newdata = test_fold)
    fold_APSE[i] <- mean(abs(test_fold$ln_brain - y_pred))
  }
  cv_error[q] <- mean(fold_APSE)
}

results <- data.frame(
  degree = 1:6,
  cv_error = cv_error
)

results
```

```
##   degree  cv_error
## 1      1 1.0296863
## 2      2 1.0656070
## 3      3 0.9200882
## 4      4 1.3540332
## 5      5 3.1400077
## 6      6 3.0437722
```

```r
which.min(cv_error)
```

```
## [1] 3
```

Model 3 has the lowest APSE, which disagrees with the result in 2c.

This comparison is not very fair since both models use different objective functions. In Q2c, we used mean squared error to evaluate the models, and in Q3d we used mean absolute error. This may result in conflicting degree selection.

e.

```r
loss_function <- function(E) {
  return(log(cosh(E)))
}

cv_error_l1 <- numeric(6)
cv_error_l2 <- numeric(6)

# L1
for (q in 1:6) {
  fold_APSE <- numeric(5)
  for (i in 1:5) {
    test_index <- folds[[i]]
    train_fold <- Animals[-test_index, ]
    test_fold <- Animals[test_index, ]

    model <- lad(ln_brain ~ poly(ln_body, q), data = train_fold)
    y_pred <- predict(model, newdata = test_fold)
    E <- test_fold$ln_brain - y_pred
    fold_APSE[i] <- mean(loss_function(E))
  }
  cv_error_l1[q] <- mean(fold_APSE)
}

# L2
for (q in 1:6) {
  fold_APSE <- numeric(5)
  for (i in 1:5) {
    test_index <- folds[[i]]
    train_fold <- Animals[-test_index, ]
    test_fold <- Animals[test_index, ]

    model <- lm(ln_brain ~ poly(ln_body, q), data = train_fold)
    y_pred <- predict(model, newdata = test_fold)
    E <- test_fold$ln_brain - y_pred
    fold_APSE[i] <- mean(loss_function(E))
  }
  cv_error_l2[q] <- mean(fold_APSE)
}

results <- data.frame(
  degree = 1:6,
  cv_error_l1 = cv_error_l1,
  cv_error_l2 = cv_error_l2
)
results
```

```
##   degree cv_error_l1 cv_error_l2
## 1      1   0.6474223   0.7850752
## 2      2   0.6006533   0.5280574
## 3      3   0.5534479   0.5925662
## 4      4   0.9197470   0.9009190
```

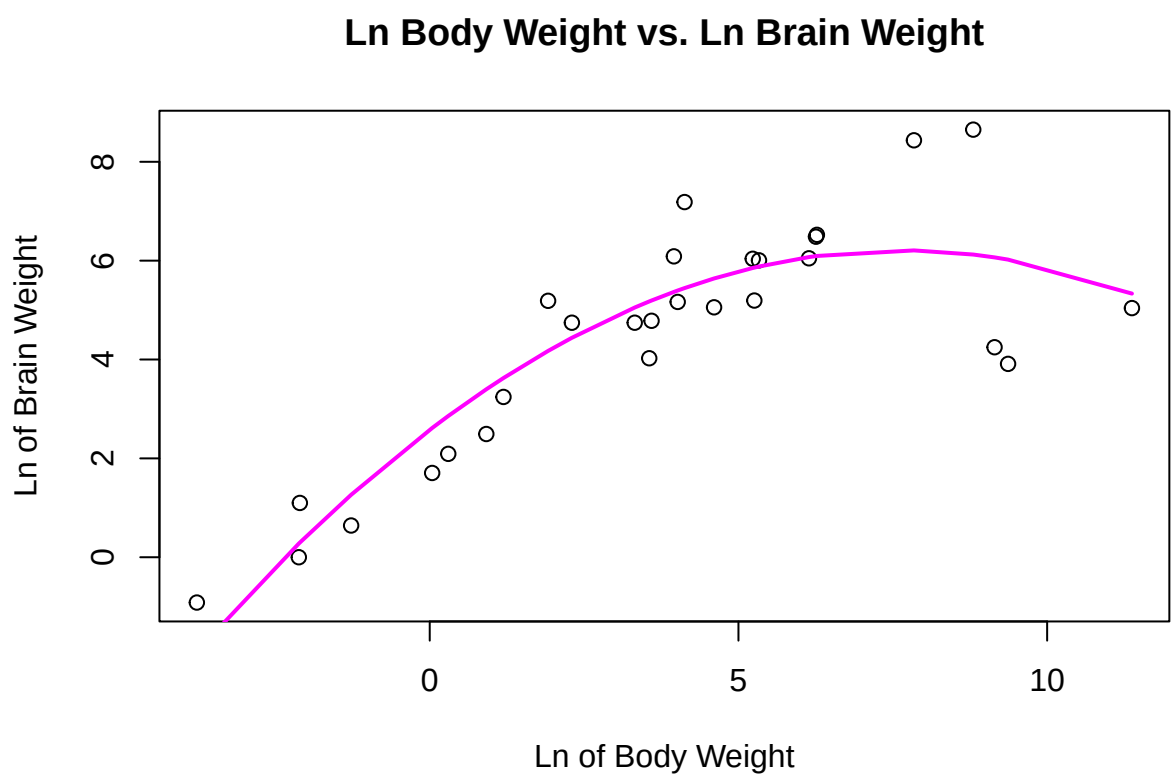
```
## 5       5   2.7093737   2.5178116
## 6       6   2.6161077   2.8532053
```

A model with degree 2 and L2 loss has the smallest log-cosh error, so we select it as the best model.

Q4.

a.

```r
model <- lm(ln_brain ~ poly(ln_body, 2), data = Animals)

plot(Animals$ln_body, Animals$ln_brain,
     xlab = "Ln of Body Weight",
     ylab = "Ln of Brain Weight",
     main = "Ln Body Weight vs. Ln Brain Weight")

lines(Animals$ln_body[ord], fitted(model)[ord], col = "magenta", lwd = 2)
```

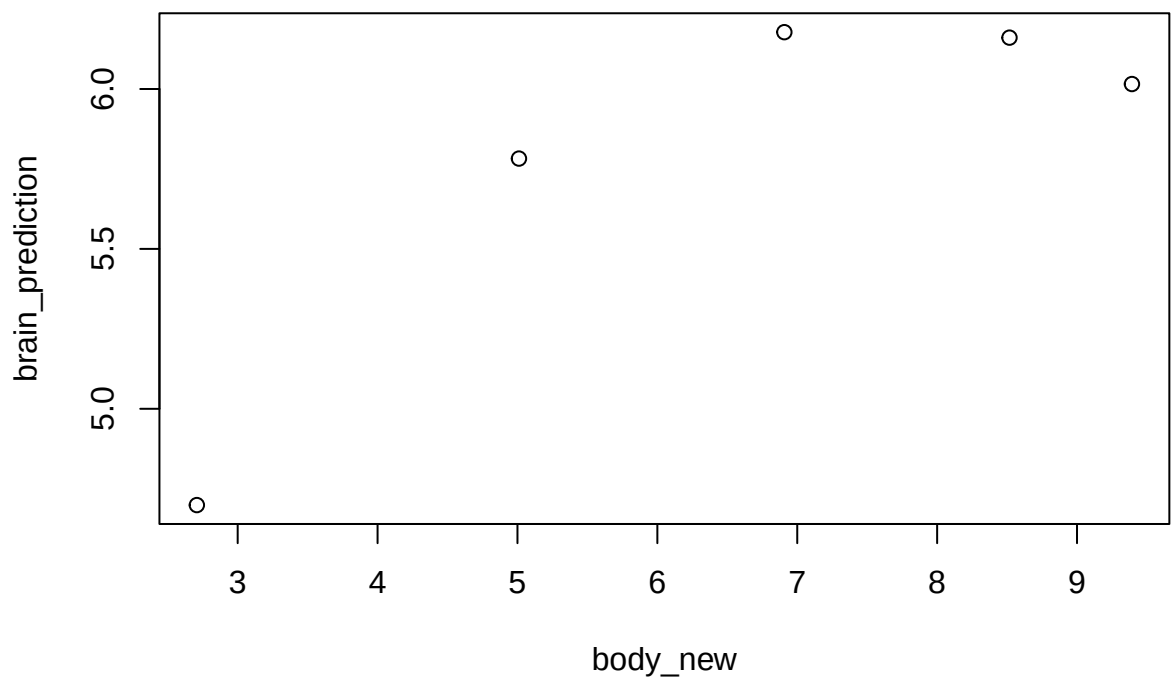


b.

```r
body_new <- log(c(15, 150, 1000, 5000, 12000))
brain_prediction <- predict(model, newdata = data.frame(ln_body = body_new))
exp(brain_prediction)
```

```
##        1        2        3        4        5
## 109.8231 324.5455 481.8809 473.7334 409.7445
```

```r
plot(body_new, brain_prediction)
```



c.

```r
max(Animals$ln_body)
```

```
## [1] 11.37366
```

```r
max(body_new)
```

```
## [1] 9.392662
```

```r
min(Animals$ln_body)
```

```
## [1] -3.772261
```

```r
min(body_new)
```

```
## [1] 2.70805
```

We see that the body weights are within the range of the data, so this is interpolation.